

5LSL0 Machine learning for signal processing

Assignment title: 1-fundamentals

Student name and contact information:

c.gheorghiu1@student.tue.nl

Gheorghiu Cristian Stefan

Exercise 1.3

- a) Write a function that takes two strings as inputs, representing paths to stored weights. The function should return two object (i.e., your denoiser and your detector with the weights loaded from the files)

Initially one function taking two strings as inputs, representing paths to stored weights was put in place. However, due to the very comprehensive nature of my class this function was cloned and adequately converted to support “modes” each meant to put in action a certain setup (e.g., **Fig 1** – training and obtaining a “better denoiser” based on a composite loss that factors in the detector; **Fig 2** – apply the denoiser on the test set to create a list of denoised images and consequently use the detector on the clean images, the noisy images and the denoised images;)

```
1 def activate_Adversial_Mode(self, path:[str, str], baseline_change=None):
2
3     # 11. Load detector backbone architecture and its weights;
4     print("Loading weights for detector...")
5     detector = Noisy_Image_Identifier_Net(activation=['LeakyReLU', 'Sigmoid'],
6     width=[32**2, 24**2, 22**2, 20**2, 22**2, 24**2, 32**2, 1])
7     detector.load_state_dict(
8         torch.load(path[0]))
9
10    # 12. Load denoiser backbone architecture and its weights;
11    print("Loading weights for denoiser...")
12    denoiser = BasicFCN(activation=['LeakyReLU', 'Tanh'],
13    width=[32**2, 24**2, 22**2, 20**2, 22**2, 24**2, 32**2])
14    denoiser.load_state_dict(
15        torch.load(path[1]))
```

Fig 1 – Better Denoiser Training Function

```
1 def activate_Super_Training_Denoiser_Mode(self, epochs, alpha=0.1, detector_path=None, denoiser_path=None):
2     """
3     Train a denoiser against the frozen Super Detector.
4
5     Objective:
6     total detector_loss:      total_loss = reconstruction_loss - alpha * detector_loss
7     BCE loss of the frozen detector when denoised images are labelled as noisy.
8     Since we subtract this term, the denoiser is trained to increase detector loss,
9     therefore fooling the detector.
10
11    Only the denoiser parameters are updated.
12    """
13
14    # 1. Defining directories and default paths;
15    self._utils_Directory_Exec()
16    if detector_path is None:
17        detector_path = r'training_Super_detector_submission\training_Super_detector\training_Super_detectormodel_weights_epoch19.pth'
18    if denoiser_path is None:
19        denoiser_path = r'train_denoiser_3_submission\train_denoiser\train_denoisermodel_weights_epoch19.pth'
```

Fig 2 – Adversial Mode

- b) Load the best version of your denoiser and the best version of your detector from the previous exercises. Apply the denoiser on the test set to create a list of denoised images. Then, apply your detector on the clean images, the noisy images, and the

denoised images. Create a figure with three boxplots, showing the likelihood of each image being noisy in each set, as estimated by your noise detector.

Fig 3 – **detector intuition on authentic clean and noisy images** – displays 3 separate boxplots showcasing how well (accurately/confidently) the model is distinguishing between classes belonging to any of these three datasets clean and noisy images, plus denoised images.

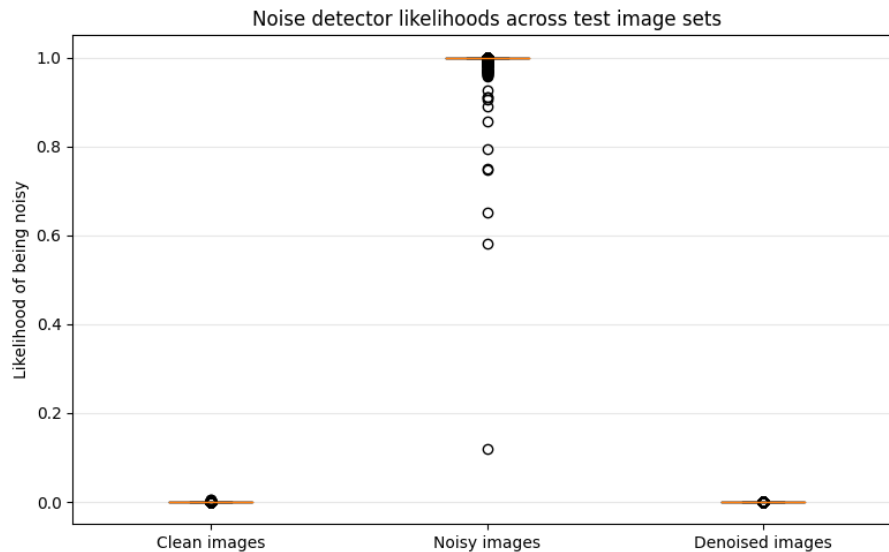


Fig 3 – Detector intuition on authentic clean and noisy images

- c) For each image, calculate whether your detector was “fooled” by your denoiser (i.e., whether the estimated likelihood changed). Make a boxplot of the changes

For sake of clarity I decided to display the **change in detector likelihood** – Fig 4 separately from the other figures.

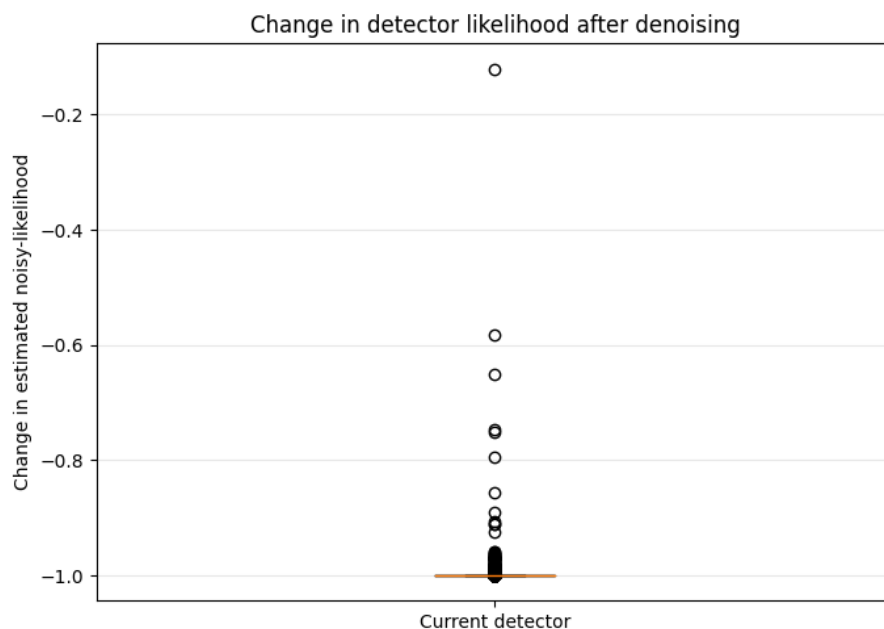


Fig 4 – Change in the detector likelihood after denoising

- d) Use `plt.subplots` to create a 3x3 grid showing the samples where your detector was “the most fooled”, “the least fooled”, and a sample that corresponds to the median change in estimated likelihood. Show the clean, noisy and denoised images, and label each image with the estimated likelihoods

Fig 5 – **Detector Fooledness** – shows what the detector was actually labelling as 0 – not noisy – or 1 – noisy. As one might expect, the figures it deems not to be corrupted with noise are not necessarily correctly labelled (e.g., the sample located at row 3, col 3 still holds quite a bit of noise yet the detector confidently asserts it’s not a noisy figure)

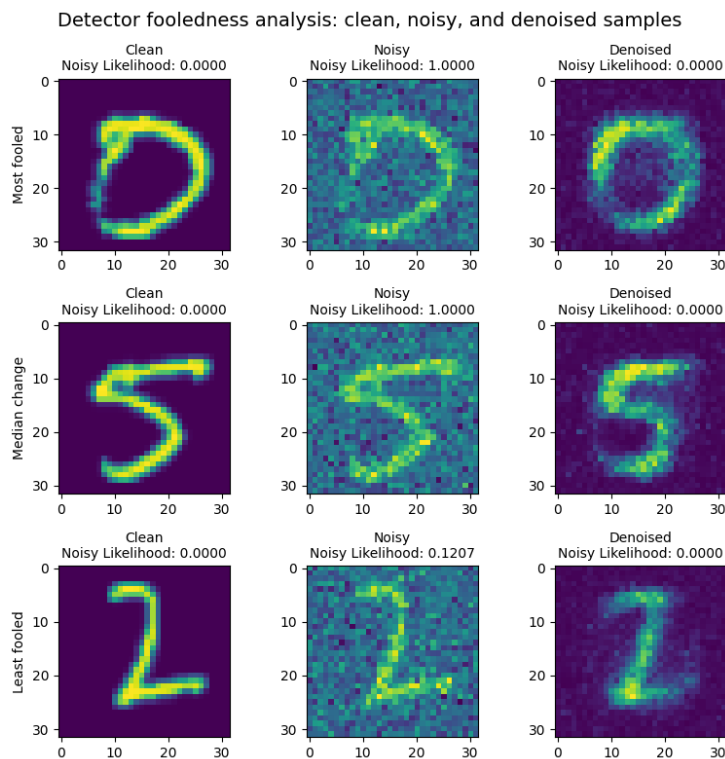


Fig 5 – Samples representing most and least fooled, plus the median change;

- e) Now use the denoiser to make a better detector! Write a new training loop for your detector that uses your denoiser as an augmentation for the training samples. 50% of the time, use the original noisy training images and 50% of the time, run the noisy inputs through your denoiser first. Remember, that your optimizer should only optimize the weights in the detector, not the denoiser.

In order to obtain a setup as described in the aforementioned requirement a different class method titled ‘**activate_Super_Training_Detector_Mode()**’ was created, which as shown in figure 6 is internally randomly augmenting the dataset and assembling (e.g., concatenating) clean images (`cl`) with noisy images (`augmented_noisy <- cl, augmented_noisy`)

```

1 def activate_Super_Training_Detector_Mode(self, epochs):
2     # 11. Creates directories & initialize backbone architecture
3     self._utils_Directory_Exec()
4     detector = Noisy_Image_Identifier_Net(
5         activation=['LeakyReLU', 'Sigmoid'],
6         width=[32**2, 24**2, 22**2, 20**2, 22**2, 24**2, 32**2, 1]
7     )
8     # 12 Load the pre-trained denoiser;
9     denoiser = BasicFCN(
10        activation=['LeakyReLU', 'Tanh'],
11        width=[32**2, 24**2, 22**2, 20**2, 22**2, 24**2, 32**2]
12    )
13    ...
14    # 14.3 Random choice 50% noisy, 50% denoised
15    # ... generates a one-dimensional tensor holding randomly generated numbers (0,1)
16    # ... then by comparison < 0.5 those numbers will be converted in boolean flags (True/False)
17    # ... then use boolean indexing to create a mixture of noisy-authentic images and reconstruction images
18    use_denoised_mask = torch.rand(current_b_size) < 0.5
19    augmented_noisy = ns.clone()
20    augmented_noisy[use_denoised_mask] = denoised_ns[use_denoised_mask]
21
22    # 14.4 Build detector batch; Shuffle batch
23    # ... Clean images -> label 1
24    # ... Noisy/denoised-noisy images -> label 0
25    detector_inputs = torch.cat((cl, augmented_noisy), dim=0)
26    clean_labels = torch.ones(current_b_size, 1)
27    noisy_labels = torch.zeros(current_b_size, 1)
28    detector_labels = torch.cat((clean_labels, noisy_labels), dim=0)
29    shuffle_indices = torch.randperm(detector_inputs.size(0))
30    detector_inputs = detector_inputs[shuffle_indices]
31    detector_labels = detector_labels[shuffle_indices]

```

Fig 6 – Class Method used to make a better detector

- f) After training, how do the final validation metric compare to the ones found in (2d)? Does that match your expectations? Explain why or why not in <= 3 sentences

In contrast with the original detector when equipping the **super detector** a tremendous improvement becomes apparent that is clearly visible in Fig 7 – **Change in detector likelihood after denoising**. By seeing augmented samples the detector became **sensible to a new “in-between” distribution** it has never learned to handle. In 2d in spite of its high accuracy score achieving around 90%/91% in this case we get a higher accuracy score (almost infinitesimal ~ 0.000154). Is indeed matching my expectations since the **Super Detector** is aware of a wider data distribution than before.

- g) Generate figures like in (3b-d) with your new detector. For the figure showing the change in estimated likelihoods, also include a boxplot with the original data from (3c) for easier comparison

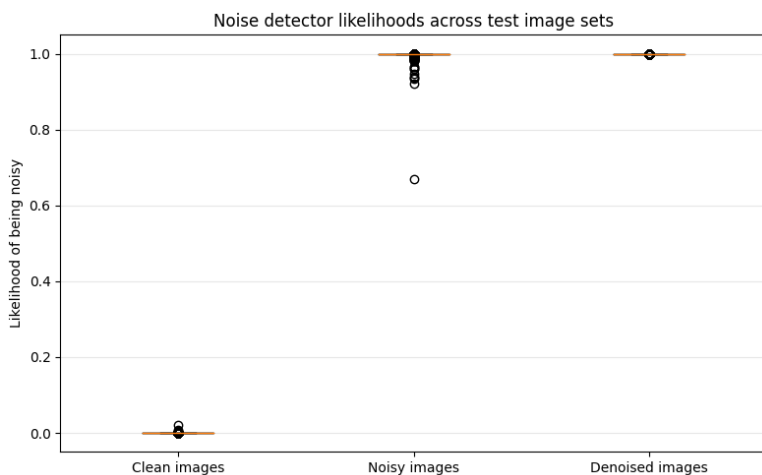


Fig 8 – Super detector intuition on authentic clean and noisy images, plus denoised images

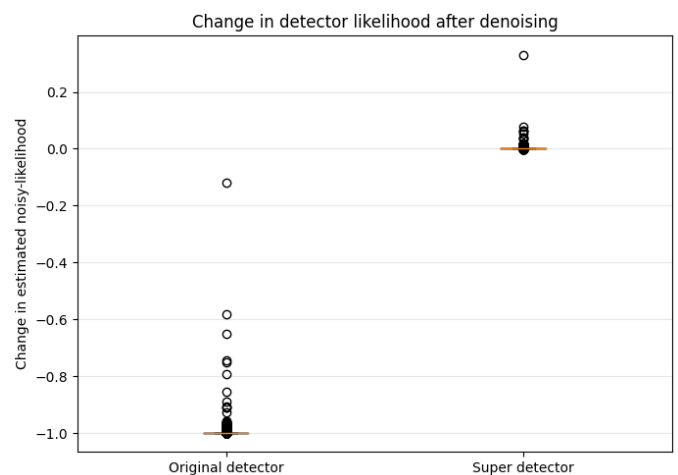


Fig 7 – Change in detector likelihood after denoising

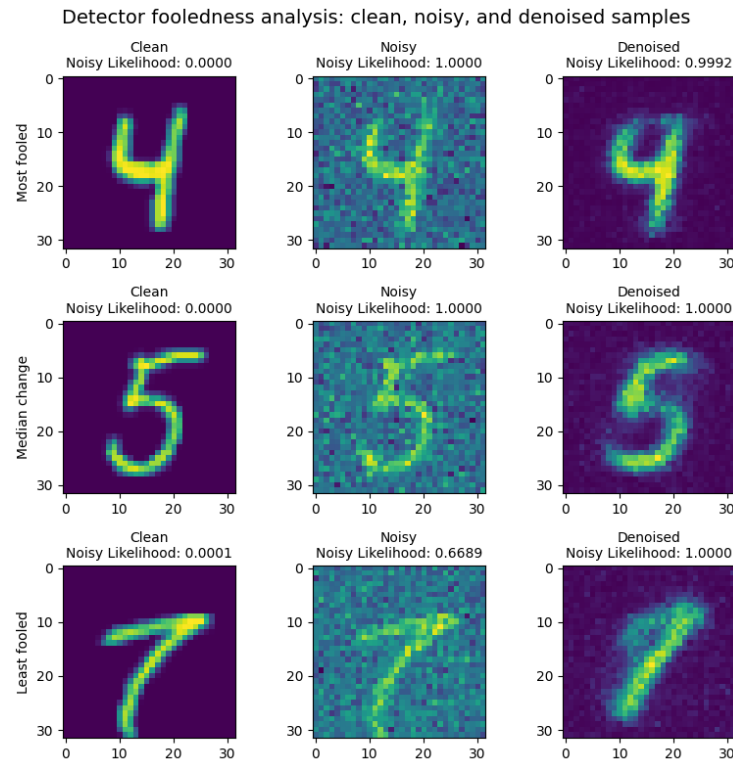


Fig 8 – Samples representing most and least fooled, plus the median change;

- Now use your new detector to make a better denoiser! Write a new training loop for your denoiser that runs the output through your detector. Compute a loss for the detector output, multiply it by $-\alpha$, and then add it to the loss of your denoiser. We want to train our denoiser to fool the detector, so the detection loss should go up, not down. Remember that your optimizer should only optimize the weights in the denoiser, not the detector. Plot both components of the loss as separate lines.
- Depending on the loss functions you chose for both of your networks, different values for α may be appropriate. Explain how you chose your α (optionally support your answer with a figure of experimental results)

First of all, for the sake of simplicity and clarity I will address both h and i points simultaneously from a programmatical standpoint (e.g., displaying and explaining the part of my code that I make use of to conduct the experiment) and analysis perspective (e.g., trying to explain the observed effect changing alpha had).

```

1 def activate_Super_Training_Denoiser_Mode(self, epochs, alpha=0.1, detector_path=None, denoiser_path=None):
2     """
3     Train a denoiser against the frozen Super Detector.
4
5     Objective:
6     total detector_loss: total_loss - reconstruction_loss - alpha * detector_loss
7     BCE loss of the frozen detector when denoised images are labelled as noisy.
8     Since we subtract this term, the denoiser is trained to increase detector loss,
9     therefore fooling the detector.
10
11     Only the denoiser parameters are updated.
12     """
13
14     # 4. Training loop
15     for epoch in range(epochs):
16         denoiser.train()
17         detector.eval()
18         epoch_reconstruction_loss = 0.0
19         epoch_detector_loss = 0.0
20         epoch_total_loss = 0.0
21         for ci, ns, label in self.train_loader:
22             denoised_images = denoiser(ns)
23             reconstruction_loss = reconstruction_loss_fn(
24                 denoised_images,
25                 ci)
26             # -----
27             # 4.1 Detector loss
28             # -----
29             # Denoised noisy images originate from noisy images,|
30             # ... so target is 0 from the detector's perspective.|
31             # ... But we subtract this loss, forcing the denoiser|
32             # ... to make detector loss go up.|
33             # -----
34             detector_output = detector(denoised_images)
35             noisy_targets = torch.zeros_like(detector_output)
36             detector_loss = detector_loss_fn(
37                 detector_output,
38                 noisy_targets)
39             # 5 Combined adversarial denoiser objective
40             total_loss = reconstruction_loss - alpha * detector_loss
41

```

Fig 9 – Class Method used to build a better Denoiser

Programming:

As shown in Fig 9 – **Class Method used to build a better Denoiser** – another class method taking as arguments the alpha level, detector_path and denoiser_path was put together for this purpose. Within the training loop the **composite loss** defined as $total_loss = reconstruction_loss - \alpha * detector_loss$ can be seen.

Experiment:

In order to carry out this experiment I adopted and tested for the following alpha values = [0.01, 0.05, 0.5, 0.8] for which four trials came about. With alpha = 0.1, the first plot in Fig 10 shows that the detector loss suddenly explodes from near 0 to roughly 90–95 around epoch 7. That means the denoiser found a way to make the detector very wrong. But that does not guarantee the image became visually better (e.g., as it happens in VAEs the model collapse is replicable within the context as defined by this example as well). From the least high alpha level (0.05) to the highest (0.8) we observe that the detector sudden loss increase takes place way earlier (the second epoch) which led to the following statement:

Higher alpha usually increases pressure to fool the detector, so detector loss may increase. However, if alpha is too large, the denoiser may sacrifice reconstruction quality to fool the detector. Therefore, better-looking reconstructions require a balanced alpha, not necessarily a larger alpha

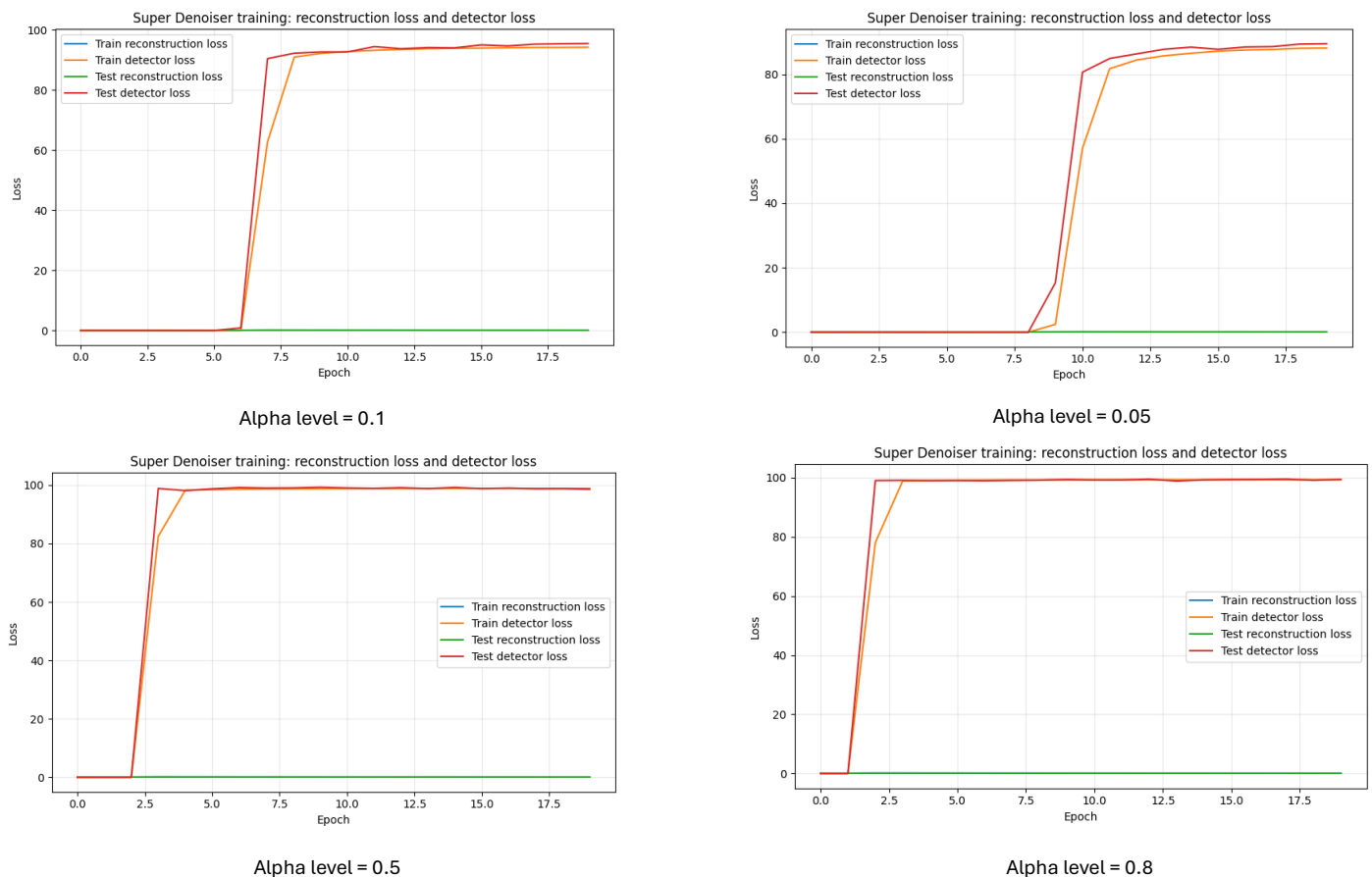


Fig 10 – Experimental results

- j) Generate figures like in (3g) with your new detector and your new denoiser. For the figure showing the change in estimated likelihoods, also include boxplots with the data from (3c) and (3g) for easier comparison.

